# A Comparison of Neural Architectures for Movie Poster Genre Classification *

Samson Lam

May 2024

**Abstract**

A movie poster is the first visual contact a viewer has with a film, and it signals genre before any other information reaches the audience. We measure how far that signal survives as a machine-readable one, and at what cost. Three regimes are compared, a convolutional network, four ImageNet backbones and YOLOv8 transferred to posters, and a vision-language model, against two backbones built for cheap inference, EfficientNetV2 and Fast-ViT. We find that the classifier draws on the whole poster rather than its center, tolerates rotation and is largely indifferent to color. Poster design changes measurably across decades, and supplying the decade as an indicator raises the accuracy by about 4 %. ResNets deliver the strongest results, reaching 54.5 % accuracy on the 19-genre task and 69.7 % on the simplified 3-genre task.[1]

## 1 Introduction

A movie poster carries a deliberate visual argument. It is composed to tell a viewer, in the moment before any synopsis is read, what kind of film this is. Genre is the part of that argument the designer most wants to land, and it is encoded in choices a human reads without effort, the palette, the typography, the arrangement of faces, the presence or absence of a horizon. Whether those choices survive as a machine-readable signal is an empirical question, and it is the question this study addresses.

We treat genre recognition as image classification on the poster alone. The dataset is the Letterboxd collection published on Kaggle (Garanin, 2024), from which 548,835 posters survive data cleaning, each labeled with one to four of 19 genres. This study compares different machine learning architectures, beginning with a convolutional neural network (CNN). VGG16 (Simonyan and Zisserman, 2015), AlexNet (Krizhevsky et al., 2012) and two depths of ResNet (He et al., 2016) supply the transfer learning. YOLOv8 (Jocher et al., 2023) contributes a detection backbone adapted to classification. CLIP (Radford et al., 2021) is evaluated zero-shot, without any training on posters, and serves as the reference a fine-tuned model has to beat. EfficientNetV2 (Tan and Le, 2021) and Fast-ViT (Vasu et al., 2023) are backbones designed for cheap inference, and they test whether the accuracy available at this task requires the parameter counts the older backbones carry.

The remainder of the report is organized as follows. Section 2 describes the dataset and the data cleaning pipeline. Section 3 trains a CNN from scratch. Section 4 covers the four transfer learning

*This is a class project for the Chinese University of Hong Kong's STAT 4012 course, completed by a group of six members.

[1]Code and replication data are available at *GitHub*.

models and YOLOv8. Section 5 covers the zero-shot and the fine-tuned efficient backbones. Section 6 collects the comparison tables and concludes.

## 2 Data

The Letterboxd collection consists of 557,539 poster images with genre, country, language and release date tables. The image size of $(345, 230, 3)$ dominates with 99.1 % of the data, as Table 19 in the appendix shows. We drop posters that do not conform to the dominant shape of $(345, 230, 3)$, leaving 552,531. Next, movies carrying an implausible number of genres are treated as outliers. $[-0.904, 4.006]$ is the range of genre count within 3 standard deviations of the mean, so we set a ceiling of four genres per movie. This step discards 3,827 movies of the genre table, 3,696 of them among the 552,531, leaving 548,835 movies.

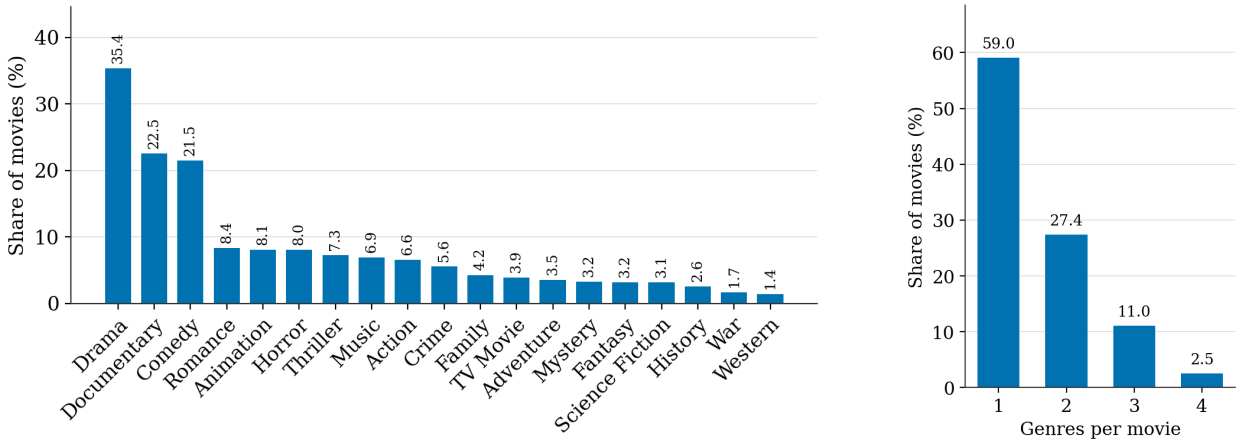



Figure 1: Distribution of genres (left) and the number of genres per movie (right).

Figure 1 displays the shares of the 19 genres on the left, with Drama, Documentary and Comedy leading, and the number of genres carried on the right, where the mass sits on one or two. Figure 2 gives the production region and the spoken language. The dataset is dominated by European and North American productions and by English-language releases. This imbalance is used in Section 4.1, where we evaluate the trained classifier separately by region and language to test its accuracy dependence.

The first release dates span from 1874 to 2031, with some posters created well in advance of the actual release date. Figure 3 illustrates their distribution by decade. On the grounds that the earlier material is sparse and stylistically distinct from the rest, the CNN of Section 3 uses only posters released after 1970 with at most three genres, leaving a total of 424,102 posters after all data cleaning.

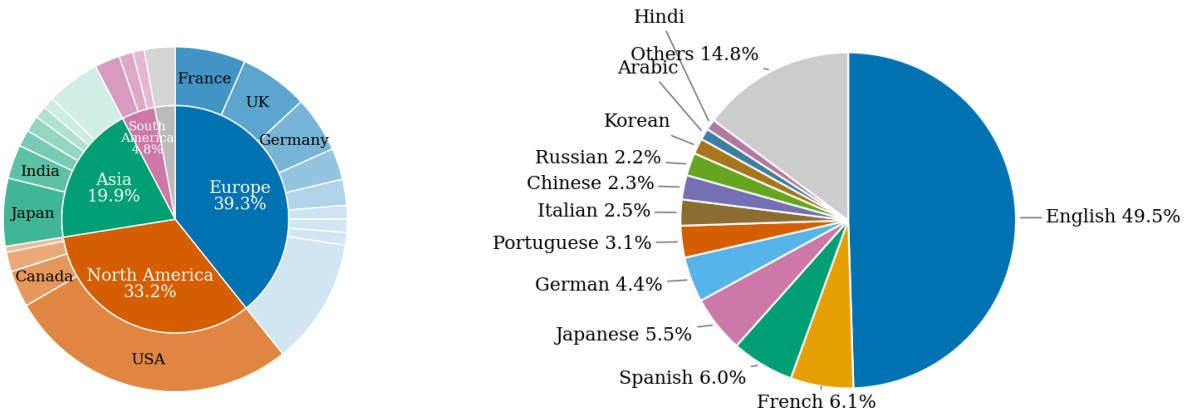



Figure 2: Distribution of regions (left) and languages (right).

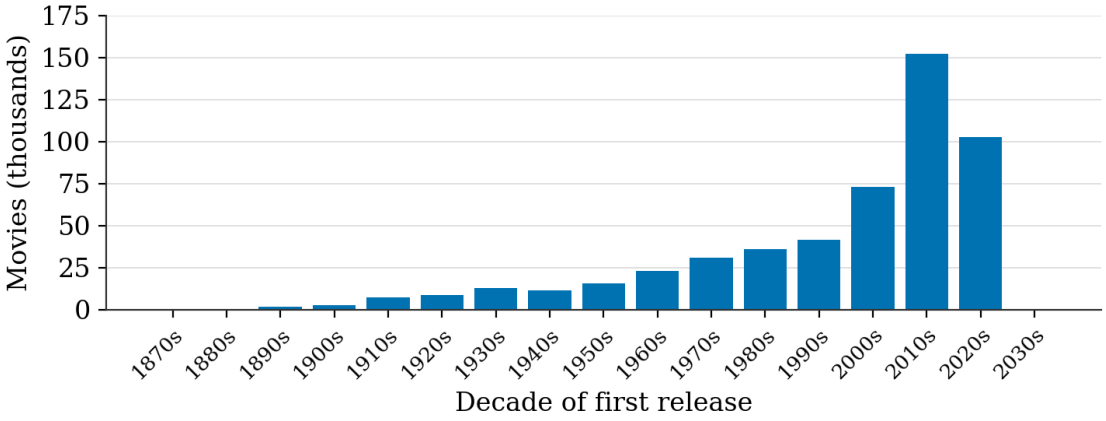



Figure 3: Movies by decade of first release.

## 3 Convolutional Neural Network

This section constructs a CNN from scratch, which fixes the floor every later model is measured against. Given the architecture limits, we remove two genres before training. Horror and Thriller rely on the same visual vocabulary, suspense, threat and psychological pressure, and are difficult to separate even by eye, so Thriller is left out. Western is also dropped due to a lack of images.

### 3.1 Model

We propose a 12-layer CNN model, with the initial 9 layers having 32 filters and the last 3 layers having 64 filters. All layers use a (3, 3) kernel, ReLU activation, and padding. Group normalization is applied after each layer to improve learning, and max-pooling layers downsample the feature maps. After flattening, batch normalization is applied, and the data passes into a dense layer with 32 units and ReLU activation. Dropout regularization with a rate of 0.25 is then employed to prevent overfitting. In the next layer, batch normalization is applied again, and the data passes through another dense layer with 64 units and ReLU activation. Dropout regularization with a rate of 0.5 is applied at this stage. Finally, a dense layer with the number of genres $G$ to classify and softmax activation is added, producing predicted probabilities for each genre. The model is compiled using the Adam optimizer, categorical cross entropy loss, and accuracy as the evaluation metric. Figure 14 in the appendix draws the stack.

The sample contains 9,000 posters: 7,200 for fitting and 1,800 for testing, balanced across genres. An initial attempt with $G = 5$ genres produced accuracy, F1 and receiver operating characteristic (ROC) curves no better than a random guess, which motivated the reduction to 3 genres reported below.

Poster design is not stationary. Figure 4 places one poster from each of four decades side by side, and the progression is visible without measurement. The 1910s poster is monochrome and relies on tonal contrast. The 1950s poster carries the vivid, broadly realistic palette Technicolor made available. By the 1990s, the colors are saturated and chosen for impact. The 2010s poster returns to restraint, using gradients and narrow tonal ranges to suggest depth. Therefore, we group movie poster release dates by decades, supplying this as an indicator.

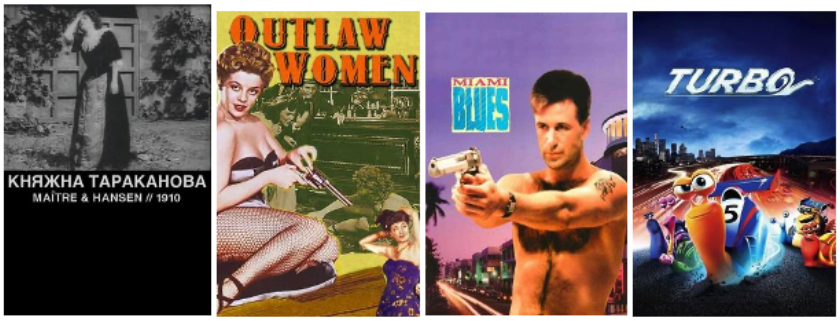



Figure 4: Poster design across four decades, from the 1910s (left) to the 2010s (right).

### 3.2 CNN Performance

We narrow our classification to 3 genres, namely [Action, Drama and Romance], then [Comedy, Drama and Romance], then [Drama, Horror and Music]. Table 1 reports the area under the ROC curve (AUROC), F1, precision and accuracy for releases after 1970 and after 2000. Across repeated runs, the indicators raise accuracy by about 4 %, which we take as confirmation that release period carries information the pixels do not supply on their own. Nevertheless, the performance is still disappointing, as it barely performs adequately on 3 genres. As a result, the remainder of the report turns to pre-trained models, where the representation is learned elsewhere, and only the classifier is fitted to posters.

| Setting | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|
| Action, Drama, Romance, after 1970 | 60.2 | 42.6 | 42.9 | 44.5 |
| Action, Drama, Romance, after 2000 | 66.2 | 46.6 | 45.4 | 44.9 |
| Comedy, Drama, Romance, after 1970 | 62.0 | 45.6 | 46.3 | 47.3 |
| Comedy, Drama, Romance, after 2000 | 62.1 | 45.6 | 45.6 | 46.5 |
| Drama, Horror, Music, after 1970 | 62.5 | 45.2 | 45.3 | 46.6 |
| Drama, Horror, Music, after 2000 | 53.0 | 41.7 | 42.4 | 41.4 |
| Average | 60.9 | 44.6 | 44.7 | 45.2 |

Table 1: Evaluation metrics for CNN, in percent.

## 4 Transfer Learning

Transfer learning reuses a representation learned on a large dataset and fits only a new classifier to the current task. The pre-trained body has already encoded edges, textures and shapes, so training is shorter, and the risk of overfitting a small sample is lower.

### 4.1 VGG16

#### 4.1.1 Model

VGG16 is a 16-layer convolutional network built from a single repeated element, the $3 \times 3$ convolution (Simonyan and Zisserman, 2015). Its design argument is that a stack of small kernels reaches the receptive field of a larger one while using fewer parameters and inserting more rectifiers along the way, so depth buys expressiveness more cheaply than width. Convolutions are grouped into five blocks, each closed by a $2 \times 2$ max pool that halves the spatial extent, while the channel count doubles from block to block up to 512. Trained on ImageNet, the resulting body is a general-purpose feature extractor, and we use it here with frozen weights so only the classifier on top is fitted to posters.

We continue with 3 genres, Action, Romance and Thriller, on releases from 2000 to 2024. After flattening, it feeds into a dense layer with 128 units and sigmoid activation, then dropout at 0.25,

then a softmax layer with 3 outputs. Sigmoid is used in the single hidden layer because the network is shallow enough that saturation is not a concern. Table 2 gives the parameter counts.

| Layer | Output shape | Parameters |
|---|---|---|
| Input | $(345, 230, 3)$ | 0 |
| VGG16 | $(10, 7, 512)$ | 14,714,688 |
| Flatten | (35,840) | 0 |
| Dense, sigmoid | (128) | 4,587,648 |
| Dropout at 0.25 | (128) | 0 |
| Output, softmax | (3) | 387 |

Table 2: Layers of the VGG16 classifier with output shapes and parameter counts.

### 4.1.2 VGG16 Performance

Fitting minimizes categorical cross-entropy with Adam at a learning rate of 0.001, in mini-batches of 32, for 50 epochs. Figure 5 plots validation accuracy and loss by epoch on the left and the ROC curves on the right, and Table 3 gives the metrics on the held-out posters. Accuracy and F1 both sit near 61 %, and the area under the curve reaches 79.0 %, a clear advance over the CNN in Section 3 at a fraction of the training cost.

| Model | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|
| VGG16 | 79.0 | 61.4 | 62.0 | 61.4 |

Table 3: VGG16 on the held-out 3-genre sample, in percent.

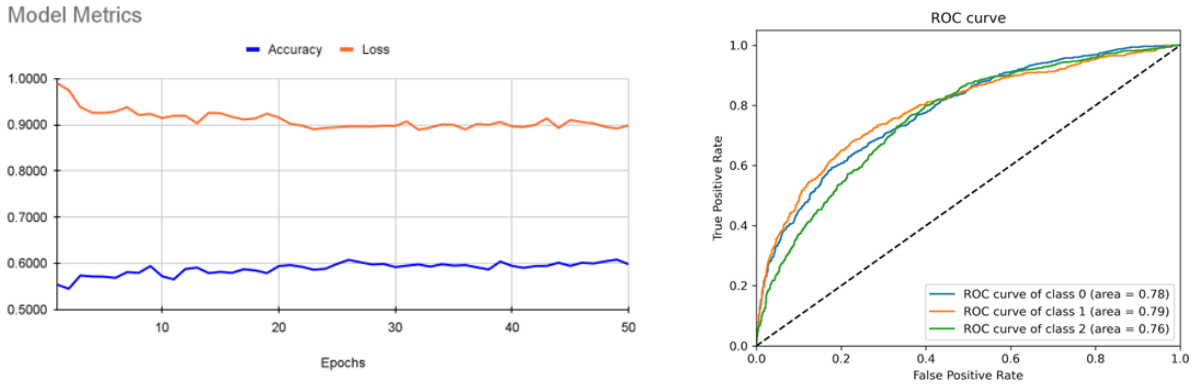



Figure 5: Fit and discrimination of the VGG16 classifier. Validation accuracy and loss by epoch on the left, ROC curves on the held-out sample on the right.

### 4.1.3 Model Inspection

While the VGG16 gives a moderately accurate result, we shall explain the classification procedures involved. We degrade the test posters in three controlled ways and re-run the trained classifier on each, holding everything else fixed and reading the F1 score. First, we inspect the importance of colors, corners and orientation. Figure 6 shows one poster under each transformation beside the original.




Figure 6: A test poster in its original form and under the three transformations, monochrome conversion, blanking of 30 border pixels and rotation by 180 degrees.

Table 4 gives the confusion matrices, each cell written as the original count followed by the count after the transformation.

| Transformation | True genre | Predicted | | |
|---|---|---|---|---|
| | | Action | Romance | Thriller |
| Monochrome, F1 = 59.4 % | Action | 482 → 409 | 134 → 165 | 184 → 226 |
| | Romance | 119 → 80 | 510 → 507 | 150 → 192 |
| | Thriller | 169 → 133 | 170 → 177 | 482 → 511 |
| Border removed, F1 = 45.7 % | Action | 482 → 202 | 134 → 60 | 184 → 538 |
| | Romance | 119 → 63 | 510 → 244 | 150 → 472 |
| | Thriller | 169 → 45 | 170 → 40 | 482 → 736 |
| Rotated, F1 = 54.9 % | Action | 482 → 526 | 134 → 94 | 184 → 180 |
| | Romance | 119 → 247 | 510 → 345 | 150 → 187 |
| | Thriller | 169 → 252 | 170 → 115 | 482 → 454 |

Table 4: Confusion matrices under three transformations of the test posters, written as the original count followed by the transformed count. The original F1 is 61.4 %.

Removing color barely registers. We test how much the absence of color hinders our classification, since a more vibrant color scheme in movie posters could affect our judgment of their genre. Hence, we converted the posters to grayscale. Averaging the three channels leaves the confusion

matrix almost intact and F1 falls only to 59.4 %, from which we infer that the network uses color chiefly to locate edges, and that the grayscale gradient preserves those edges.

Removing the border does considerable damage. We test how much information the border of the posters contains for the classification, since it seems logical that the center of the posters contains the most crucial information about the movies. Hence, we blanked 30 pixels along each edge, which surprisingly drives F1 down to 45.7 % and pushes most posters into the Thriller column. The classifier is therefore not concentrating on the center of the poster in the way a human viewer does. Its filters act across the whole image and any part removed is a loss.

Turning the poster upside down costs little. F1 holds at 54.9 % and the loss is concentrated in Romance. We attribute this to Romance posters carrying human contact more often than the other two genres, and the reading of a contact between two figures is sensitive to orientation in a way that a skyline or a weapon is not, which is a plausible account of why that genre alone suffers.

Next, we examine whether the classifier has cultural biases, since the dataset is dominated by Western and English-language releases. Tables 5 and 6 evaluate it separately by production region and by spoken language. F1 is lowest for African and South American releases, which also have the fewest examples. Across languages, the spread is narrower, and Hindi is the only clear shortfall, while Korean releases score above the English baseline. Oversampling underrepresented regions and languages can help, but we do not attempt it here.

| | Africa | Asia | Europe | North America | South America | Oceania |
|---|---|---|---|---|---|---|
| F1 | 54.7 | 61.3 | 60.3 | 64.8 | 57.6 | 59.2 |

Table 5: VGG16 F1 by production region, in percent.

| | English | Spanish | Cantonese | Hindi | Korean | Thai |
|---|---|---|---|---|---|---|
| F1 | 61.5 | 62.1 | 61.4 | 57.0 | 65.8 | 61.6 |

Table 6: VGG16 F1 by spoken language, in percent.

To gain insights into the operation, we analyze the feature maps. Reading the activations directly confirms the usual picture of a convolutional hierarchy. As the model progresses through its convolutional layers, it extracts more complex features from the images. Layer 8 responds to edges and layer 16 responds to shapes and finer structure, and on a poster for Ip Man it isolates the face. Figure 7 gives the extract.

Overall, VGG16 is therefore rotation tolerant, largely indifferent to color, but reliant on the whole image rather than its center. However, it does not appear to capture the spatial relation between objects, which is where the deeper and more recent architectures in the following sections earn their place.

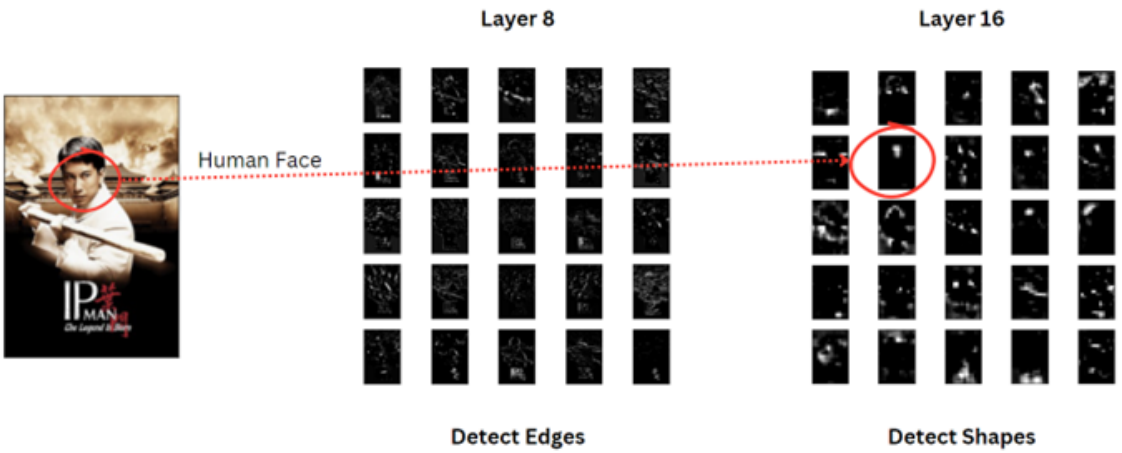



Figure 7: Feature maps from layers 8 and 16 on a poster for Ip Man.

## 4.2 AlexNet and Residual Networks

While Sections 3 and 4.1 reduced the problem to $G = 3$ genres, this section returns to the full 19-genre problem.

### 4.2.1 The Models

AlexNet is the network that made deep convolutional models the default for image classification, winning the 2012 ImageNet competition at a top-one accuracy of 63.3 % and by a wide margin over the runner-up (Krizhevsky et al., 2012). Five convolutional layers feed 3 fully connected ones, with rectifier activations throughout and dropout in the classifier. The implementation used here sets the channels to 64, 192, 384, 256 and 256 and carries 61.1 million parameters, almost all of them in the two dense layers of width 4,096. By current standards, it is shallow and is included as the weakest of the pre-trained bodies rather than as a competitive candidate.

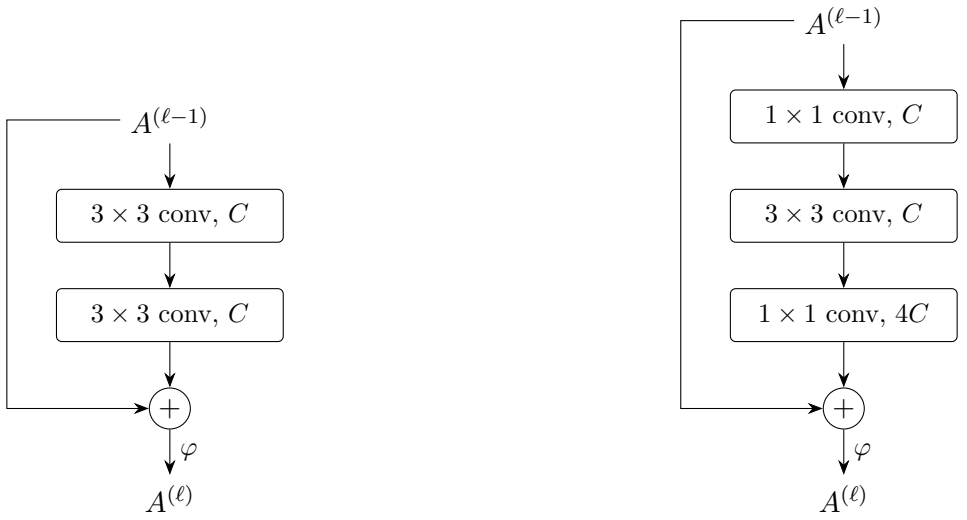
Residual networks (ResNets) answer the problem that depth alone creates. Past a certain number of layers, accuracy degrades even on the training set, so the difficulty lies in optimization rather than overfitting. The remedy is to have each block learn a residual against its own input rather than a full transformation,

$$A^{(\ell)} = \varphi\Big(\mathcal{F}\Big(A^{(\ell-1)}; \theta^{(\ell)}\Big) + A^{(\ell-1)}\Big), \tag{1}$$

where $\mathcal{F}(\cdot\,; \theta^{(\ell)})$ is the stack of convolutions inside block $\ell$, so the identity is available at no cost (He et al., 2016).

Figure 8 draws the two block types the two depths use. ResNet-18 stacks the basic block, two $3 \times 3$ convolutions of equal width. ResNet-50 stacks the bottleneck block, a $1 \times 1$ convolution that narrows the channels, a $3 \times 3$ convolution at that reduced width and a $1 \times 1$ convolution that widens by a factor of four, which buys depth at a fraction of the arithmetic. Table 7 sets out the three backbones. Figures 15 and 16 in the appendix draw the AlexNet stack and the residual stages. The

pair of depths is what makes the comparison informative. If depth is what matters the two should separate, and if the residual connection is what matters both should stand clear of AlexNet.




(a) Basic block, ResNet-18.

(b) Bottleneck block, ResNet-50.

Figure 8: The two residual blocks of Equation (1).

| | AlexNet | ResNet-18 | ResNet-50 |
|---|---|---|---|
| Stem | $11 \times 11$ conv, stride 4 | $7 \times 7$ conv, stride 2 | $7 \times 7$ conv, stride 2 |
| Body | five convolutions | 2, 2, 2, 2 basic | 3, 4, 6, 3 bottleneck |
| Widths | 64 to 384 | 64 to 512 | 256 to 2,048 |
| Head | two dense, 4,096 | global pool, linear | global pool, linear |
| Parameters | 61.1 M | 11.7 M | 25.6 M |

Table 7: The three transfer learning backbones, as implemented in torchvision.

#### 4.2.2 AlexNet and ResNet Performance

All three backbones take the poster resized to $224 \times 224$. We replace each model's softmax output with a linear layer with sigmoid activation, since a movie may carry several genres, and we use binary cross-entropy as the loss. Training runs for 5 epochs. We compare Adam and stochastic gradient descent, with the latter at a learning rate of $10^{-4}$, while Adam uses $10^{-6}$ for AlexNet, $5 \times 10^{-5}$ for ResNet-18, and $10^{-5}$ for ResNet-50. We track multi-label AUROC during training and validation.

Adam converges faster and to a lower loss than stochastic gradient descent for both architectures. For AlexNet, the training losses are close, but the validation metrics separate clearly. For ResNets, the gap is even wider, as shown in Figure 9. Every result reported below uses Adam.

Figure 10 compares the three architectures under Adam. ResNet outperforms AlexNet throughout, consistent with greater depth allowing more abstract features and the skip connection in

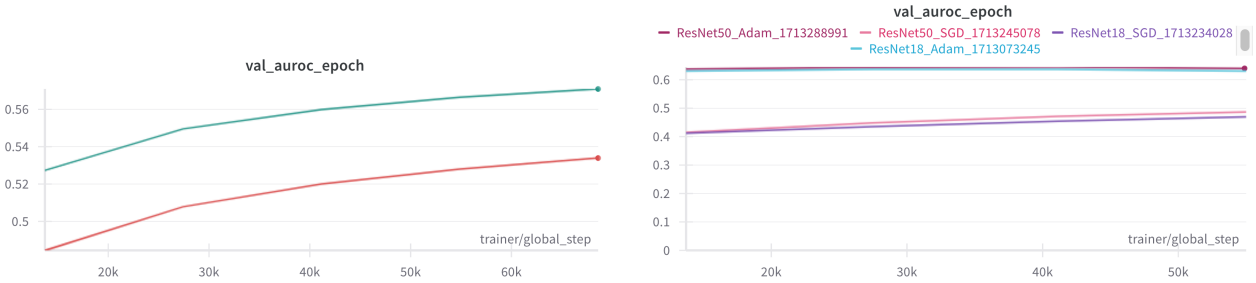



Figure 9: Validation AUROC by epoch under Adam and stochastic gradient descent, for AlexNet (left) and ResNet (right).

Equation (1) keeping the gradient usable through that depth.

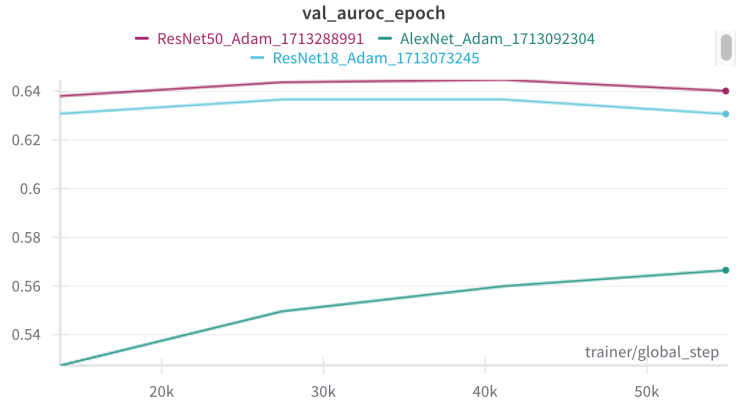



Figure 10: Validation AUROC by epoch for AlexNet and ResNets under Adam.

Table 8 reports the 19-genre task and Table 9 the 3-genre task. ResNet clearly outperforms AlexNet, but the depth within the residual family does not improve significantly. ResNet-50 is far deeper than ResNet-18, yet their training and validation curves nearly coincide, and ResNet-50 leads by a slim margin on every genre. The gap between AlexNet and ResNet is therefore attributable to the residual connection rather than to depth alone. Overall, ResNet-18 with shallower architecture performs almost as well in terms of evaluation metrics, and better in training efficiency. Table 20 in the appendix gives AUROC by genre.

| Model | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|
| AlexNet | 54.5 | 13.4 | 22.7 | 42.9 |
| ResNet-18 | 61.6 | 28.1 | 34.0 | 53.6 |
| ResNet-50 | 62.6 | 29.9 | 33.6 | 54.5 |

Table 8: AlexNet and ResNet on 19-genre single-label classification, in percent.

| Model | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|
| AlexNet | 71.6 | 61.5 | 62.8 | 61.8 |
| ResNet-18 | 77.4 | 69.8 | 69.7 | 69.7 |
| ResNet-50 | 77.5 | 69.7 | 70.2 | 69.7 |

Table 9: AlexNet and ResNet on single-label classification over Action, Romance and Thriller, in percent.

## 4.3 YOLOv8

### 4.3.1 The Model

YOLO, short for You Only Look Once, is a state-of-the-art family of real-time object detectors for images and video, built for detection rather than classification. It divides the image into a grid and predicts bounding boxes with class probabilities for each cell in a single forward pass, which makes it fast enough for video. The eighth release ships a classification branch, YOLOv8n-cls, in which the detection head is replaced by a classifier over the whole image (Jocher et al., 2023). Its backbone alternates five stride-two convolutions with four C2f blocks, each of which splits its input, passes one part through a chain of bottleneck units and concatenates the results, and the head is a global pool followed by a single linear layer. The nano variant used here is the smallest of five sizes, at 2.7 million parameters against the 61.1 million of AlexNet. It is included to test whether a backbone shaped by a detection objective, which must localize objects rather than summarize a scene, reads a poster differently from a classification backbone.

### 4.3.2 YOLOv8 Performance

The input convention is a directory tree, with posters filed under a folder named for their label and split into training and validation folders. We implement this using the pre-trained YOLOv8n-cls model in two ways. Under the multi-label approach, a poster is copied into one folder per genre it carries. Under the multi-class approach, it appears once, under a single genre. Both train for 10 epochs with Adam and the default cross-entropy loss.

The multi-label approach barely learns. Over 10 epochs, the training loss falls by 0.2, and accuracy rises by 0.04, settling near 35 %, and the normalized confusion matrix in Figure 11 shows that it separates nothing except Drama. Filing one poster under several conflicting labels appears to be the cause. The multi-class approach trains more cleanly, as Figure 12 shows.

Nevertheless, Table 10 shows the multi-label model ahead on 19 genres and level or ahead on 3 genres, despite its weaker training curve. A plausible reading is that the multi-class model learns one class deeply while the multi-label model acquires broader but shallower knowledge of the label space.

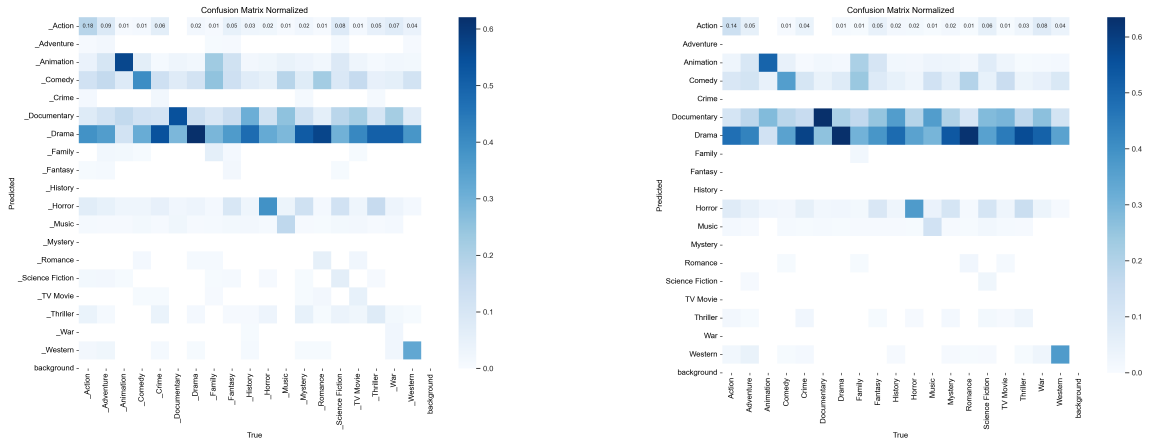



Figure 11: Normalized confusion matrices, multi-label (left) and multi-class (right).

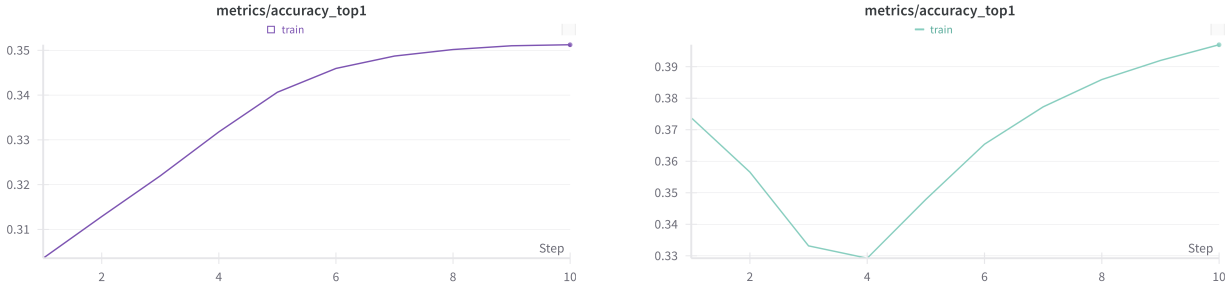



Figure 12: Top-one accuracy by epoch, multi-label (left) and multi-class (right).

| Task | Model | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|---|
| 19 genres | YOLOv8, multi-label | 82.8 | 29.7 | 29.7 | 29.7 |
| | YOLOv8, multi-class | 82.0 | 29.0 | 29.0 | 29.0 |
| 3 genres | YOLOv8, multi-label | 60.4 | 39.6 | 39.6 | 39.6 |
| | YOLOv8, multi-class | 53.8 | 39.6 | 39.6 | 39.6 |

Table 10: The two YOLOv8 models on single-label classification, in percent.

# 5 Zero-Shot Prediction and Efficient Backbones

## 5.1 CLIP

CLIP is a multi-modal model trained to align images and text in a shared embedding space (Radford et al., 2021). Write $f_I$ for the image encoder, $f_T$ for the text encoder and

$$s_{ij} = \frac{f_I(X_i)^\top f_T(T_j)}{\|f_I(X_i)\| \, \|f_T(T_j)\|} \tag{2}$$

for the cosine similarity between the $i$th image and the $j$th caption of a batch of $N$ matched pairs. Pre-training is contrastive. It maximizes the similarity of the $N$ matched pairs while minimizing it across the $N^2 - N$ mismatched ones, by minimizing

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2N} \sum_{i=1}^{N} \left[ \log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^{N} \exp(s_{ij}/\tau)} + \log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^{N} \exp(s_{ji}/\tau)} \right], \tag{3}$$

with $\tau > 0$ a learned temperature. Because the two encoders share one embedding space, a classification task can be posed without any further training by writing each label as a sentence $T_g$ and returning

$$\hat{y}(X) = \underset{g \in \{1,\dots,G\}}{\arg\max} \frac{f_I(X)^\top f_T(T_g)}{\|f_I(X)\| \, \|f_T(T_g)\|}, \tag{4}$$

as Figure 13 sets out. Figure 17 in the appendix places that step inside the contrastive pre-training of Equation (3).

That property makes CLIP the natural reference point for this study. It has never seen the poster dataset, so whatever it achieves is the accuracy available with no task-specific training, and a fine-tuned model has to beat it to justify its cost.

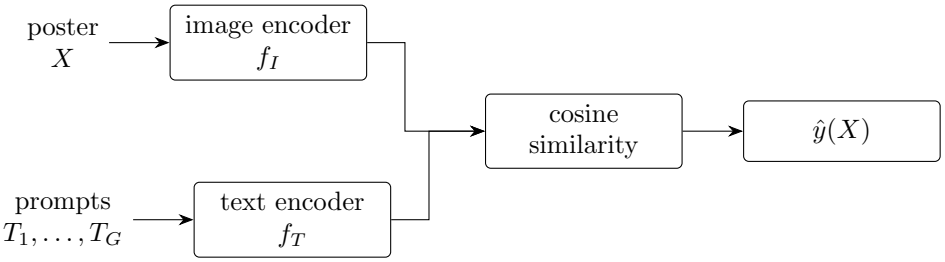



Figure 13: Zero-shot classification with CLIP. Each genre is written as one sentence, both sides are embedded, and Equation (4) returns the genre whose sentence lies closest to the poster.

The label set is written as the prompt *A movie poster of* {} *genre*, one sentence per genre, which is tokenized and passed through the text encoder. The poster is passed through the image encoder. The model returns a probability across the prompts, so an output of $[0.3, 0.3, 0.4]$ over Comedy, Documentary and Drama reads as a 0.4 probability of Drama. Four image encoders are evaluated, namely ResNet-50, ResNet-50x4 at roughly four times the arithmetic, and the base

vision transformer at patch sizes 32 and 16, ViT-B/32 and ViT-B/16, which stacks 12 transformer blocks of width 768 with 12 attention heads over the flattened patches (Dosovitskiy et al., 2021), drawn in Figure 18 of the appendix. The text encoder is held fixed across the four.

### 5.2 EfficientNetV2

EfficientNet scales a network along depth, width and input resolution together rather than one at a time (Tan and Le, 2019). Depth adds layers and so enlarges the receptive field, width adds feature maps, and resolution retains more of the image. The three are tied to a single compound coefficient $\phi$,

$$d = \alpha^{\phi}, \qquad w = \beta^{\phi}, \qquad r = \gamma^{\phi}, \qquad \text{subject to} \quad \alpha\beta^2\gamma^2 \approx 2, \quad \alpha, \beta, \gamma \geq 1, \tag{5}$$

where $d$, $w$ and $r$ are the depth, width and resolution multipliers. The constraint is what makes the scaling economical, since the arithmetic grows linearly in depth but quadratically in width and in resolution, so a unit increase in $\phi$ raises the floating-point operations by a factor of about two whichever way the budget is spent.

The second version adds progressive learning (Tan and Le, 2021), which begins training at a small image size under weak regularization and raises both as training proceeds, so that the expensive large-image passes occur only once the model is ready for them. Its body mixes two block types. The fused inverted bottleneck, which merges the expansion and the depthwise convolution into one ordinary convolution, is used in the early stages where the feature maps are large, and the standard inverted bottleneck is used later. Table 11 gives the stage structure.

### 5.3 Fast-ViT

Vision transformers are slow and memory inefficient. Splitting an image of resolution $r$ into patches of side $p$ yields a sequence of $n = (r/p)^2$ tokens, and self-attention costs $O(n^2)$ in that length, so halving the patch size multiplies the attention cost by a factor of 16. Fast-ViT is a hybrid design that keeps convolutions wherever a transformer would be expensive (Vasu et al., 2023). It is staged like a residual network, halving resolution and doubling channels from one stage to the next, it replaces patch embedding with a convolutional stem, and it mixes tokens with convolutions in place of self-attention. That substitution rests on the MetaFormer observation that the transformer block keeps working when self-attention is replaced by another token mixer, a spatial multi-layer perceptron or even plain pooling (Yu et al., 2022), which Figure 19 in the appendix draws. Its own token mixer, RepMixer, uses a $3 \times 3$ depthwise convolution with batch normalization, reparameterized into a single convolution once training is complete. The variant used here is T12, with 12 blocks, the second smallest of the family after T8, in which every stage mixes tokens this way and no attention layer remains.

| | EfficientNetV2-S | Fast-ViT T12 |
|---|---|---|
| Stem | $3 \times 3$ conv, 24 channels | $3 \times 3$ convolutions, stride 4 |
| Stage widths | 24, 48, 64, 128, 160, 256 | 64, 128, 256, 512 |
| Blocks per stage | 2, 4, 4, 6, 9, 15 | 2, 2, 6, 2 |
| Block type | fused and standard bottleneck | RepMixer |
| Head | $1 \times 1$ conv to 1,280, linear | $1 \times 1$ conv to 1,024, linear |
| Parameters | 21.5 M | 7.6 M |

Table 11: The two fine-tuned backbones of this section, as implemented in torchvision and in timm.

### 5.4 Fine-Tuning with Time Embedding

Both models are fine-tuned in the same way. We load the pre-trained weights, freeze the body so only the head is estimated, and replace the head that outputs 1,000 ImageNet classes with one that outputs the $G$ genres. Write $\mathbf{z} = \mathcal{A}(f_{\text{body}}(X)) \in \mathbb{R}^q$ for the globally averaged feature vector, with $q = 1{,}280$ for EfficientNetV2 and $q = 1{,}024$ for Fast-ViT. For Fast-ViT, the head is a single affine map to $G$ outputs. For EfficientNetV2 it is two affine maps separated by dropout,

$$\boldsymbol{\eta} = U_2 \,\mathrm{Drop}_{0.2}(U_1 \,\mathrm{Drop}_{0.2}(\mathbf{z}) + \mathbf{c}_1) + \mathbf{c}_2, \qquad \Pr(y = g \mid X) = \frac{\exp(\eta_g)}{\sum_{g'=1}^{G} \exp(\eta_{g'})}, \tag{6}$$

with $U_1$ of width 1,000. Training minimizes cross-entropy using Adam for either 10 or 22 epochs.

The time embedding variant adds the release year. The year enters as a one-hot vector $\boldsymbol{\iota}(t) \in \{0,1\}^K$ over the $K$ distinct release years of the dataset, each coordinate of which indexes a learned four-dimensional embedding, so the year contributes a vector $\mathbf{u}(t) \in \mathbb{R}^{4K}$. It is concatenated with the pooled features, so that the head of Equation (6) receives $\left(\mathbf{z}^\top, \mathbf{u}(t)^\top\right)^\top$ in place of $\mathbf{z}$ and $U_1$ widens to $q + 4K$ columns.

### 5.5 Performance

Both loaders were constructed over the entire frame, so the rows used for evaluation are the rows the models were fine-tuned on. Two label sets are reported, the full 19 genres and a reduced set of 3 genres [Comedy, Documentary and Drama]. Table 12 restricts the dataset by release period. Accuracy improves slightly as the window narrows, indicating a real but small time-varying element.

| Image encoder | All periods | After 1970 | After 2000 |
|---|---|---|---|
| ResNet-50 | 25.5 | 25.7 | 26.4 |
| ViT-B/32 | 31.7 | 33.6 | 34.4 |

Table 12: Zero-shot accuracy of CLIP over 19 genres by release period, in percent.

Tables 13 and 14 compare the four image encoders. The ordering follows encoder strength on 19 genres, with the patch-16 vision transformer ahead throughout.

| Image encoder | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|
| ResNet-50 | 75.9 | 16.7 | 19.3 | 25.5 |
| ResNet-50x4 | 78.2 | 18.4 | 20.4 | 25.5 |
| ViT-B/32 | 79.3 | 19.0 | 20.4 | 31.7 |
| ViT-B/16 | 81.3 | 21.0 | 22.0 | 32.9 |

Table 13: Zero-shot CLIP over 19 genres, in percent.

| Image encoder | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|
| ResNet-50 | 74.4 | 52.8 | 55.8 | 53.2 |
| ResNet-50x4 | 71.6 | 45.4 | 51.2 | 51.0 |
| ViT-B/32 | 74.3 | 51.3 | 56.3 | 55.1 |
| ViT-B/16 | 79.8 | 60.0 | 60.5 | 60.9 |

Table 14: Zero-shot CLIP over Comedy, Documentary and Drama, in percent.

Tables 15 and 16 give the fine-tuned results against the best zero-shot figure. The time embedding hurts rather than helps EfficientNetV2 at both epoch counts, so we did not carry it over to Fast-ViT. On 19 genres, EfficientNetV2 leads Fast-ViT, plausibly because the 12-block Fast-ViT is a small variant of its family, and both lead the zero-shot reference, which is what fine-tuning is supposed to deliver. On 3 genres, zero-shot CLIP edges ahead of both in 20 minutes against 100 and 135, which suggests that on an easy label set the textual description of the genre is worth more than task-specific fitting.

| Model | Epochs | Without time embedding | With time embedding |
|---|---|---|---|
| EfficientNetV2 | 10 | 45.6 | 44.1 |
| EfficientNetV2 | 22 | 46.2 | 30.4 |
| Fast-ViT T12 | 10 | 36.9 | |
| Fast-ViT T12 | 22 | 39.8 | |
| CLIP, zero-shot | | 32.9 | |

Table 15: Accuracy after fine-tuning over 19 genres, against the best zero-shot encoder, in percent.

| Model | Epochs | Accuracy | Training and inference time |
|---|---|---|---|
| EfficientNetV2 | 22 | 59.2 | 135 minutes |
| Fast-ViT T12 | 22 | 60.8 | 100 minutes |
| CLIP, zero-shot | | 60.9 | 20 minutes |

Table 16: Accuracy after fine-tuning over Comedy, Documentary and Drama, against the best zero-shot encoder, in percent.

## 6 Conclusion

To conclude, Tables 17 and 18 display the performance of the models scored on all four metrics, and Tables 15 and 16 give the accuracy of the fine-tuned backbones. We believe a movie poster carries enough genre signal to be worth modeling but not enough to assign a reliable label. Training a CNN from scratch is not worth the compute, since transfer learning achieves better results in a fraction of the time. We found that the drift in poster design is a real but small time-varying element in the data. The model inspection showed that VGG16 is rotation tolerant, largely indifferent to color, but reliant on the whole image rather than its center, which is a different reading strategy from the one a human viewer brings to the same picture. Across the comparison, ResNets generally achieve the best results. YOLOv8 and CLIP with ViT-B/16 also lead on AUROC on 19 and 3 genres respectively, and demonstrate the capability of more complex and efficient architectures.

| Model | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|
| CNN, average | 60.9 | 44.6 | 44.7 | 45.2 |
| VGG16 | 79.0 | 61.4 | 62.0 | 61.4 |
| AlexNet | 71.6 | 61.5 | 62.8 | 61.8 |
| ResNet-18 | 77.4 | **69.8** | 69.7 | **69.7** |
| ResNet-50 | 77.5 | 69.7 | **70.2** | **69.7** |
| YOLOv8, multi-label | 60.4 | 39.6 | 39.6 | 39.6 |
| YOLOv8, multi-class | 53.8 | 39.6 | 39.6 | 39.6 |
| CLIP, ResNet-50 | 74.4 | 52.8 | 55.8 | 53.2 |
| CLIP, ResNet-50x4 | 71.6 | 45.4 | 51.2 | 51.0 |
| CLIP, ViT-B/32 | 74.3 | 51.3 | 56.3 | 55.1 |
| CLIP, ViT-B/16 | **79.8** | 60.0 | 60.5 | 60.9 |

Table 17: Evaluation metrics on 3-genre single-label classification, in percent. The CNN row averages the settings of Table 1, VGG16 to YOLOv8 use Action, Romance and Thriller, and CLIP uses Comedy, Documentary and Drama.

| Model | AUROC | F1 | Precision | Accuracy |
|---|---|---|---|---|
| AlexNet | 54.5 | 13.4 | 22.7 | 42.9 |
| ResNet-18 | 61.6 | 28.1 | **34.0** | 53.6 |
| ResNet-50 | 62.6 | **29.9** | 33.6 | **54.5** |
| YOLOv8, multi-label | **82.8** | 29.7 | 29.7 | 29.7 |
| YOLOv8, multi-class | 82.0 | 29.0 | 29.0 | 29.0 |
| CLIP, ResNet-50 | 75.9 | 16.7 | 19.3 | 25.5 |
| CLIP, ResNet-50x4 | 78.2 | 18.4 | 20.4 | 25.5 |
| CLIP, ViT-B/32 | 79.3 | 19.0 | 20.4 | 31.7 |
| CLIP, ViT-B/16 | 81.3 | 21.0 | 22.0 | 32.9 |

Table 18: Evaluation metrics on 19-genre single-label classification, in percent.

## References


A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. 9th Int. Conf. Learn. Represent. (ICLR)*, 2021.

S. Garanin, 2024, "Letterboxd (Movies Dataset)," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/gsimonx37/letterboxd

K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8. (2023). Ultralytics. [Online]. Available: https://docs.ultralytics.com/models/yolov8/

A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 25, 2012, pp. 1097–1105.

A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 8748–8763.

K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015.

M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 10096–10106.

P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, "FastViT: A fast hybrid vision transformer using structural reparameterization," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 5762–5772, doi: 10.1109/ICCV51070.2023.00532.

W. Yu et al., "MetaFormer is actually what you need for vision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 10809–10819, doi: 10.1109/CVPR52688.2022.01055.

# A Appendix

## A.1 Poster Dimensions

Table 19 gives the distribution of image dimensions over the 557,540 poster files of the raw download, one of which cannot be read, from which Section 2 keeps only the dominant shape.

| Image size | Count | Share (%) |
|---|---|---|
| $(345, 230, 3)$ | 552,531 | 99.1 |
| $(345, 230)$ | 4,203 | 0.8 |
| $(344, 230, 3)$ | 752 | 0.1 |
| $(344, 230)$ | 32 | 0.0 |
| Other sizes | 21 | 0.0 |
| Invalid format | 1 | 0.0 |

Table 19: Poster dimensions with counts and shares of the corpus.

## A.2 Model Architectures

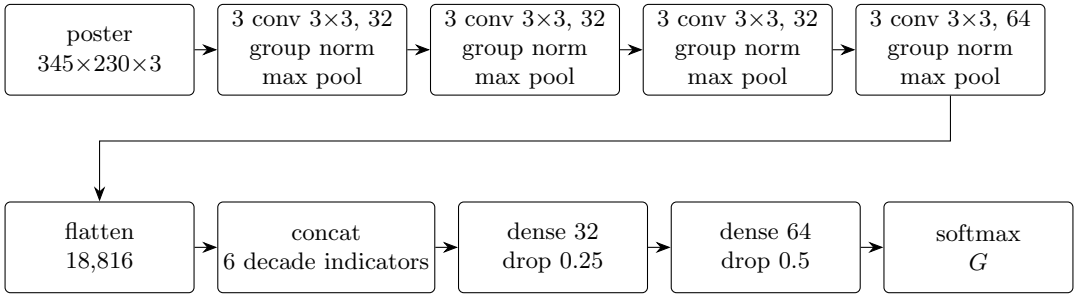



Figure 14: The convolutional network built from scratch of Section 3.

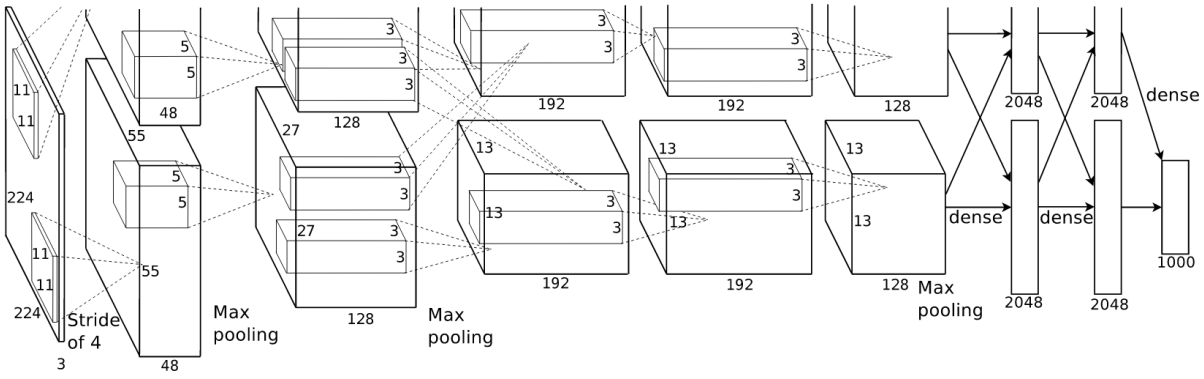



Figure 15: AlexNet, reproduced from Krizhevsky et al. (2012).

| layer name | output size | 18-layer | 34-layer | 50-layer | 101-layer | 152-layer |
|---|---|---|---|---|---|---|
| conv1 | 112×112 | 7×7, 64, stride 2 | | | | |
| conv2_x | 56×56 | 3×3 max pool, stride 2 | | | | |
| | | $\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$ |
| conv3_x | 28×28 | $\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$ | $\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$ |
| conv4_x | 14×14 | $\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$ | $\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$ |
| conv5_x | 7×7 | $\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$ | $\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ | $\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$ |
| | 1×1 | average pool, 1000-d fc, softmax | | | | |
| FLOPs | | $1.8\times10^9$ | $3.6\times10^9$ | $3.8\times10^9$ | $7.6\times10^9$ | $11.3\times10^9$ |

Figure 16: The residual architectures, reproduced from He et al. (2016).

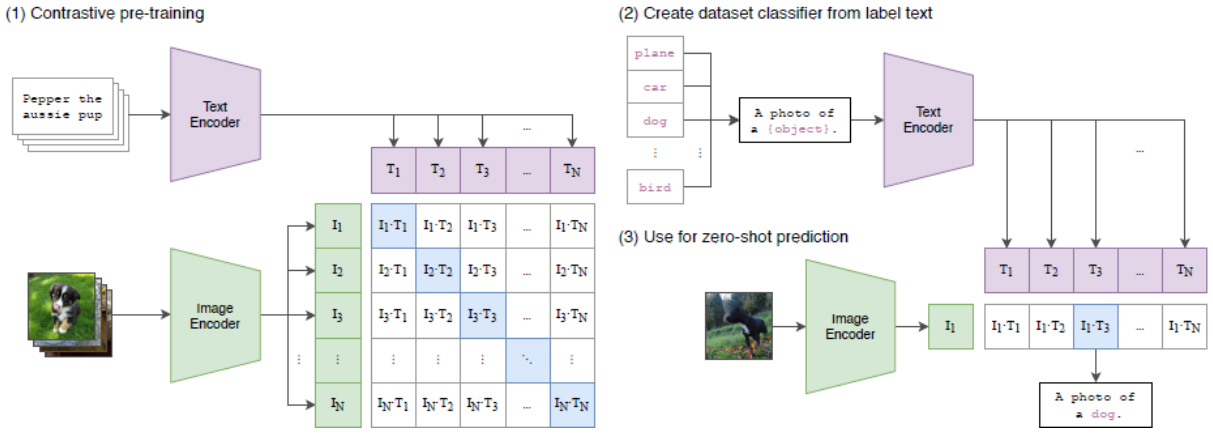



Figure 17: CLIP, reproduced from Radford et al. (2021). Panel one is the contrastive pre-training of Equation (3) and panel three is the zero-shot rule of Equation (4).

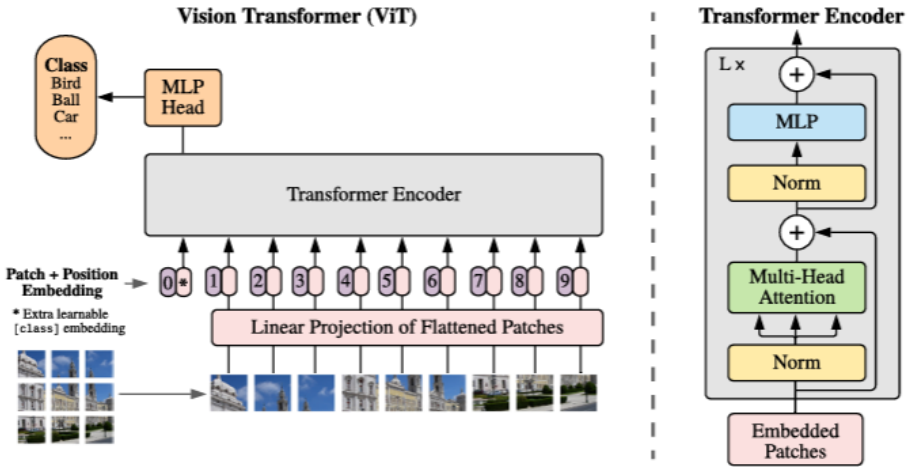



Figure 18: The vision transformer, reproduced from Dosovitskiy et al. (2021).

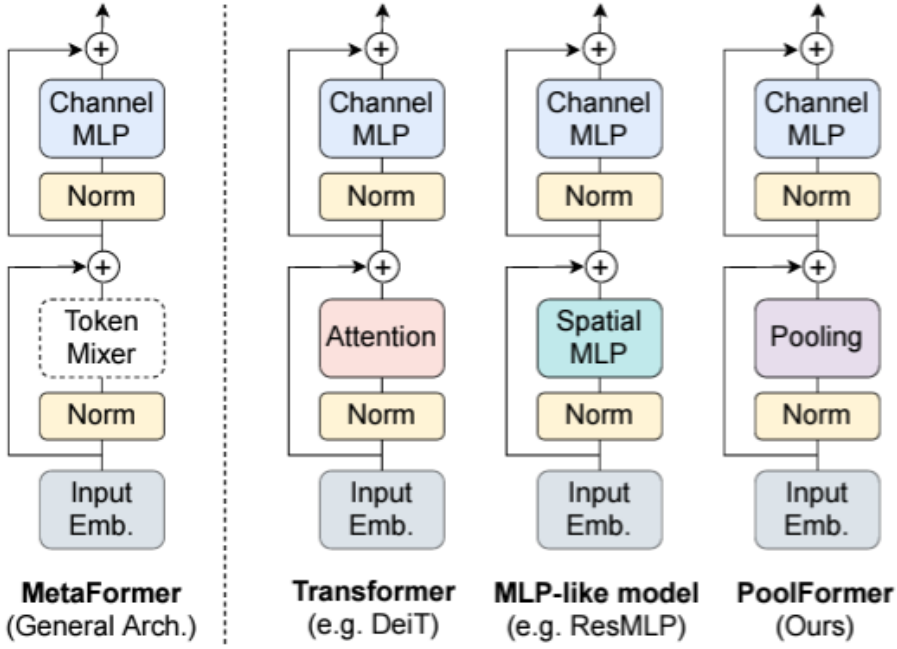



Figure 19: The MetaFormer block, reproduced from Yu et al. (2022).

## A.3 AUROC by Genre

| Genre | AlexNet | ResNet-18 | ResNet-50 |
|---|---|---|---|
| Action | 77.3 | 83.2 | 84.3 |
| Adventure | 72.1 | 77.1 | 78.6 |
| Animation | 87.9 | 94.5 | 95.2 |
| Comedy | 72.6 | 78.0 | 78.4 |
| Crime | 69.1 | 75.7 | 77.8 |
| Documentary | 75.9 | 83.4 | 83.7 |
| Drama | 66.6 | 74.6 | 75.0 |
| Family | 75.6 | 80.3 | 82.3 |
| Fantasy | 66.4 | 72.3 | 74.9 |
| History | 65.2 | 74.8 | 76.8 |
| Horror | 81.6 | 87.0 | 88.0 |
| Music | 67.6 | 78.2 | 80.0 |
| Mystery | 66.0 | 72.4 | 73.9 |
| Romance | 72.6 | 78.9 | 80.0 |
| Science Fiction | 72.5 | 78.6 | 81.0 |
| TV Movie | 70.0 | 78.2 | 80.1 |
| Thriller | 74.9 | 80.2 | 81.4 |
| War | 70.9 | 79.7 | 82.3 |
| Western | 83.8 | 92.1 | 93.3 |

Table 20: AUROC by genre on the raw scores of the test half for AlexNet and ResNet, in percent. Table 8 scores the hard prediction instead, hence its lower AUROC.

## A.4 Sample Posters by Decade

The pool is restricted to Hong Kong releases. The drift in poster design that Section 3 measures is visible here without measurement, from painted illustration and hand-set type in the earliest decades to photographic composition in the most recent.




Figure 20: 10 Hong Kong releases sampled from the 1950s.




Figure 21: 10 Hong Kong releases sampled from the 1960s.




Figure 22: 10 Hong Kong releases sampled from the 1970s.




Figure 23: 10 Hong Kong releases sampled from the 1980s.




Figure 24: 10 Hong Kong releases sampled from the 1990s.




Figure 25: 10 Hong Kong releases sampled from the 2000s.




Figure 26: 10 Hong Kong releases sampled from the 2010s.




Figure 27: 10 Hong Kong releases sampled from the 2020s.